

Lattice Tree Algorithm and Trinomial Tree Approximation

Suman Mishra

2nd October 2025

The Lattice Tree Algorithm

The lattice tree method approximates the price process S_t over multiple periods by a binomial tree, using discrete time intervals on a continuous time model. Given the initial asset price S_0 , the price after one period moves either up to S_0e^v or down to S_0e^{-v} .

After n periods, the asset price is expressed as

$$S_n = S_0 e^{X_{n,1} + X_{n,2} + \dots + X_{n,n}},$$

where each $X_{n,i}$ independently takes the value v or $-v$. By the Central Limit Theorem, the sum

$$Y = \sum_{i=1}^n X_{n,i}$$

converges in distribution to a normal variable as $n \rightarrow \infty$.

Recall under the risk-neutral measure Q , the Black-Scholes price process is

$$S_T = S_0 e^{\left(r - \frac{\sigma^2}{2}\right)T + \sigma W_T^Q},$$

where $W_T^Q \sim \mathcal{N}(0, T)$. Since

$$\left(r - \frac{\sigma^2}{2}\right)T + \sigma W_T^Q$$

is normal with mean $\left(r - \frac{\sigma^2}{2}\right)T$ and variance $\sigma^2 T$, the multi-period binomial tree approximates the Black-Scholes model by choosing suitable parameters v and risk-neutral probabilities q_u, q_d :

$$S_T \approx S_0 e^{X_{n,1} + \dots + X_{n,n}}.$$

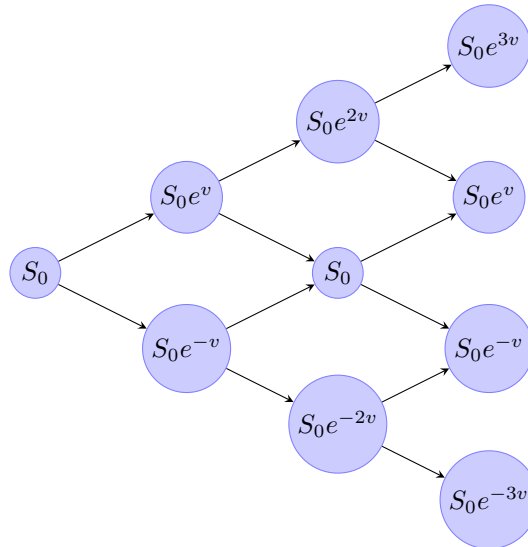


Figure 1: Multi-period binomial lattice tree for asset prices

Parameter Determination in the Lattice Tree

It remains to determine the parameters v, q_u, q_d . Based on the above analysis, we choose the parameters to match the expected value and variance of the return.

$$Y_T = \left(r - \frac{\sigma^2}{2}\right) T + \sigma W_T^Q,$$

i.e.,

$$\begin{aligned} \underbrace{\left(r - \frac{\sigma^2}{2}\right) T}_{\mathbb{E}^Q[Y_T]} &= \underbrace{n [q_u v + (1 - q_u)(-v)]}_{\mathbb{E}^Q[X_{n,1} + \dots + X_{n,n}]}, \\ \underbrace{\sigma^2 T}_{\text{Var}(Y_T)} &= n \underbrace{\left[q_u v^2 + (1 - q_u)(-v)^2 - \left(\left(r - \frac{\sigma^2}{2}\right) \frac{T}{n} \right)^2 \right]}_{\text{Var}(X_{n,1} + \dots + X_{n,n})}. \end{aligned}$$

The total return Y_T is the sum of n independent increments:

$$Y_T = \sum_{i=1}^n X_{n,i},$$

with each increment $X_{n,i}$ being

$$X_{n,i} = \begin{cases} v, & \text{with probability } q_u, \\ -v, & \text{with probability } 1 - q_u. \end{cases}$$

The expectation of one increment is

$$\mathbb{E}[X_{n,1}] = q_u v + (1 - q_u)(-v) = (2q_u - 1)v,$$

which matches the mean under Black Scholes:

$$\mathbb{E}[X_{n,1}] = \left(r - \frac{\sigma^2}{2}\right) \frac{T}{n}.$$

Thus,

$$(2q_u - 1)v = \left(r - \frac{\sigma^2}{2}\right) \frac{T}{n}.$$

The variance of one increment is

$$\text{Var}(X_{n,1}) = \mathbb{E}[X_{n,1}^2] - (\mathbb{E}[X_{n,1}])^2 = v^2 - \left(\left(r - \frac{\sigma^2}{2}\right) \frac{T}{n} \right)^2,$$

since

$$\mathbb{E}[X_{n,1}^2] = q_u v^2 + (1 - q_u)(-v)^2 = v^2,$$

and

$$(\mathbb{E}[X_{n,1}])^2 = ((2q_u - 1)v)^2 = \left(\left(r - \frac{\sigma^2}{2}\right) \frac{T}{n} \right)^2.$$

Because increments are independent, the variance of the sum is

$$\text{Var}(Y_T) = n \text{Var}(X_{n,1}) = n \left[v^2 - \left(\left(r - \frac{\sigma^2}{2}\right) \frac{T}{n} \right)^2 \right].$$

Matching with the BS Model variance gives

$$\sigma^2 T = n \left[v^2 - \left(\left(r - \frac{\sigma^2}{2}\right) \frac{T}{n} \right)^2 \right].$$

When n is sufficiently large, the term $\left(\frac{T}{n}\right)^2$ is very small and can be neglected, so the equations simplify to

$$\sigma^2 T \approx nv^2,$$

which implies

$$v = \sigma \sqrt{\frac{T}{n}} = \sigma \sqrt{\Delta t},$$

$$q_u = \frac{1}{2} + \frac{1}{2v} \left(r - \frac{\sigma^2}{2} \right) \Delta t,$$

and

$$q_d = 1 - q_u.$$

Binomial Approximation Justification

The binomial lattice approximation can be rigorously justified using the Central Limit Theorem (CLT) for triangular arrays:

Theorem (CLT for Triangular Arrays). For each n , let $Y_{n,1}, \dots, Y_{n,n}$ be independent random variables with zero mean:

$$\mathbb{E}[Y_{n,m}] = 0,$$

and suppose that

$$\sum_{m=1}^n \text{Var}(Y_{n,m}) \rightarrow \sigma^2 > 0,$$

and for every $\varepsilon > 0$,

$$\sum_{m=1}^n \mathbb{E} \left[Y_{n,m}^2 \mathbf{1}_{\{|Y_{n,m}| > \varepsilon\}} \right] \rightarrow 0,$$

as $n \rightarrow \infty$. Then the sum

$$S_n = \sum_{m=1}^n Y_{n,m}$$

converges in distribution to the normal distribution $\mathcal{N}(0, \sigma^2)$.

Verification in the binomial lattice context:

Define the centered increment

$$Y_{n,m} := X_{n,m} - \mathbb{E}[X_{n,m}] = X_{n,m} - \left(r - \frac{\sigma^2}{2} \right) \frac{T}{n}.$$

- *Why does $Y_{n,m}$ have zero mean by construction?*

By definition, the expectation operator is linear, so

$$\mathbb{E}[Y_{n,m}] = \mathbb{E}[X_{n,m} - \mathbb{E}[X_{n,m}]] = \mathbb{E}[X_{n,m}] - \mathbb{E}[X_{n,m}] = 0.$$

This centering step ensures that the random variables $Y_{n,m}$ satisfy the zero-mean condition required by the CLT.

- The total variance satisfies

$$\sum_{m=1}^n \text{Var}(Y_{n,m}) = n \text{Var}(X_{n,m}) \rightarrow \sigma^2 T,$$

as ensured by the appropriate choice of parameters v and q_u .

Explanation of the Second condition:

The second condition, (also called the Lindeberg condition) ensures that no single increment $Y_{n,m}$ has a disproportionately large impact on the sum S_n as $n \rightarrow \infty$. Formally, it requires that for every $\varepsilon > 0$,

$$\sum_{m=1}^n \mathbb{E} [Y_{n,m}^2 \mathbf{1}_{\{|Y_{n,m}| > \varepsilon\}}] \rightarrow 0.$$

In the binomial lattice case, each increment $Y_{n,m}$ takes values approximately equal to $\pm v - \mu_n$, where

$$v = \sigma \sqrt{\frac{T}{n}}, \quad \mu_n = \left(r - \frac{\sigma^2}{2} \right) \frac{T}{n}.$$

Both v and μ_n tend to zero as $n \rightarrow \infty$. Therefore, for any fixed $\varepsilon > 0$, eventually all increments satisfy

$$|Y_{n,m}| \leq v + |\mu_n| < \varepsilon,$$

which means the indicator function $\mathbf{1}_{\{|Y_{n,m}| > \varepsilon\}}$ is zero for sufficiently large n . Consequently, the Lindeberg sum becomes zero in the limit, satisfying the condition.

This guarantees that the variance is not dominated by rare large jumps, and thus the CLT applies.

Exercise 1: Trinomial Tree Approximation of Log>Returns

We now approximate the log-return X over Δt by a trinomial variable X_i taking values

$$X_i = \begin{cases} v, & \text{with probability } q_u, \\ 0, & \text{with probability } q_m, \\ -v, & \text{with probability } q_d, \end{cases}$$

where

$$v = \lambda \sigma \sqrt{\Delta t}, \quad \lambda > 0,$$

with probabilities summing to 1.

Matching the first two moments of $X \sim \mathcal{N}(\mu \Delta t, \sigma^2 \Delta t)$, where $\mu = r - \frac{\sigma^2}{2}$, leads to

$$\mathbb{E}[X_i] = v(q_u - q_d) = \mu \Delta t,$$

$$\text{Var}[X_i] \approx v^2(q_u + q_d) = \sigma^2 \Delta t,$$

ignoring terms of order $(\Delta t)^2$.

Setting

$$x := q_u - q_d = \frac{\mu \sqrt{\Delta t}}{\lambda \sigma}, \quad y := q_u + q_d = \frac{1}{\lambda^2},$$

we find

$$q_m = 1 - \frac{1}{\lambda^2}, \quad q_u = \frac{y+x}{2} = \frac{1}{2\lambda^2} + \frac{\mu \sqrt{\Delta t}}{2\lambda \sigma}, \quad q_d = \frac{y-x}{2} = \frac{1}{2\lambda^2} - \frac{\mu \sqrt{\Delta t}}{2\lambda \sigma}.$$

The corresponding price jumps are

$$u = e^v, \quad m = 1, \quad d = e^{-v}.$$

Why $\lambda = \sqrt{3}$ is Optimal: Kurtosis Matching

Step 1: Calculate Kurtosis of the Trinomial Approximation

The kurtosis κ is defined as

$$\kappa = \frac{\mathbb{E}[(X_i - \mathbb{E}[X_i])^4]}{(\text{Var}[X_i])^2}.$$

Approximating $\mathbb{E}[X_i] \approx 0$, the fourth moment is

$$\mathbb{E}[X_i^4] = v^4(q_u + q_d) = v^4 \cdot \frac{1}{\lambda^2} = \lambda^4 \sigma^4 (\Delta t)^2 \cdot \frac{1}{\lambda^2} = \lambda^2 \sigma^4 (\Delta t)^2.$$

The variance squared is

$$(\text{Var}[X_i])^2 = (\sigma^2 \Delta t)^2 = \sigma^4 (\Delta t)^2.$$

Therefore,

$$\kappa = \frac{\lambda^2 \sigma^4 (\Delta t)^2}{\sigma^4 (\Delta t)^2} = \lambda^2.$$

Step 2: Match to Continuous Model Kurtosis

The continuous GBM log-returns have kurtosis 3. To match, set

$$\lambda^2 = 3 \implies \lambda = \sqrt{3}.$$

This choice ensures the trinomial model better replicates the tail behavior of the continuous model.

Coding Implementation

Here is the Python implementation of the trinomial tree pricing algorithm. Since trinomial tree preserves previous round results, we can use a vectorized method to reduce computational time and create an efficient algorithm.

```
1 import numpy as np
2 import math
3 import time
4
5 def trinomial_tree_call_vectorized(S0, K, r, sigma, T, n, lam):
6     """
7     Compute European call option price using a vectorized trinomial tree.
8
9     Parameters:
10     S0      : initial stock price
11     K       : strike price
12     r       : risk-free interest rate
13     sigma   : volatility of the underlying asset
14     T       : time to maturity (in years)
15     n       : number of time steps in the trinomial tree
16     lam     : scaling parameter for tree step size (usually sqrt(3))
17
18     Returns:
19     option price at time zero
20     """
21
```

```

22     dt = T / n # length of each time step
23     v = lam * sigma * np.sqrt(dt) # log-price step size
24
25     # Risk-neutral probabilities for up, down, and middle moves
26     pu = 1/(2 * lam**2) + (r - 0.5 * sigma**2) * np.sqrt(dt) / (2 * lam *
sigma)
27     pd = 1/(2 * lam**2) - (r - 0.5 * sigma**2) * np.sqrt(dt) / (2 * lam *
sigma)
28     pm = 1 - 1/(lam**2)
29
30     discount = np.exp(-r * dt) # discount factor per time step
31
32     ST = S0 * np.exp(v * np.arange(-n, n + 1))
33
34     # European Call Option payoff at maturity
35     option_values = np.maximum(ST - K, 0)
36
37     # Backward induction through the tree to find option price at time zero
38     for step in range(n, 0, -1):
39         option_values = discount * (
40             pu * option_values[2:] + # up move
41             pm * option_values[1:-1] + # middle move
42             pd * option_values[:-2] # down move
43         )
44
45     # Return the option price at the root of the tree
46     return option_values[0]
47
48 # Set lambda to sqrt(3) for kurtosis matching
49 lambd = math.sqrt(3)
50
51 results = []
52 for exp in range(5, 15):
53     n = 2**exp # doubling each iteration
54     start = time.time()
55     price = trinomial_tree_call_vectorized(S0, K, r, sigma, T, n, lambd)
56     elapsed = time.time() - start
57     # Calculate percentage error compared to Monte Carlo price
58     percentage_error = abs(price - mc_price_call) / mc_price_call * 100
59     results.append((n, price, percentage_error, elapsed))
60
61 # Print results header
62 print(f"\nComparison to Monte Carlo Price (call price = {mc_price_call:.6f
}): \n")
63 print(f"{'n':>6} {'Price':>12} {'Percent Error':>12} {'Time (s)':>10}")
64
65 # Print each row of results
66 for n, price, err, t in results:
67     print(f"{n:6d} {price:12.6f} {err:12.6e} {t:10.4f}")

```

Listing 1: Trinomial Tree Option Pricing in Python

Results

Table 1: Comparison to Monte Carlo Price (call price = 5.897461)

n	Price	Percent Error	Time (s)
32	5.880680	0.284547	0.0013
64	5.900808	0.056750	0.0011
128	5.901281	0.064783	0.0018
256	5.902668	0.088300	0.0052
512	5.900990	0.059850	0.0098
1024	5.898193	0.012421	0.0214
2048	5.898531	0.018152	0.0456
4096	5.899144	0.028548	0.0974
8192	5.899117	0.028086	0.2758
16384	5.899289	0.031004	1.0067

Observations

- The trinomial tree method converges quickly to the Monte Carlo price as the number of time steps increases using the vectorized method.
- Even for relatively small numbers of steps (e.g., $n = 256$), the percentage error remains low, indicating good accuracy.
- Computational time grows with increasing n , but the vectorized implementation keeps it manageable.
- Monte Carlo simulation provides a robust benchmark price but requires a large number of trials to reduce standard error.
- The trinomial tree is more useful when pricing american options however, compared to Monte Carlo. So further studied would be required on that.

References

- [1] Professor Leung Chi Man, *MATH4514 Financial Economics in Actuarial Science Lecture Notes*, pp. 170–175.
- [2] Yue-Kuen Kwok, *Mathematical Models of Financial Derivatives*, Springer, 2008.
- [3] AI Assistance for LaTeX formatting of the Lattice Tree